Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Ders İngilizce yapıldı.
  - Asıl anlatım yaklaşık 00:09:00'da başlar.
  - Araya iki ara verildi (yaklaşık 00:51:30-01:08:20 ve 01:54:40-02:05:30).
- PDA tekrarı: anlık betimleme, hesaplama ağacı, dizgi çekme/itme kısaltması, 7'li tanım, PDA'nın dili.
- CFG'den PDA'ya dönüştürme (CFL $\subseteq$ PDA dilleri)
  - Üç durumlu genel yapı ve döngüdeki geçişler.
  - Örnek: $S \to aSTb \mid b$, $T \to Ta \mid \varepsilon$ ve $abab$ girişinin hesaplama ağacı.
  - Üretilen PDA en soldan türetimi (leftmost derivation) taklit eder.
- PDA'dan CFG'ye dönüştürme (ters yön): derste yapılmadı, kitaptan okunacak.
- Kitabın 2. bölümü bitti. Deterministik bağlamdan bağımsız diller atlandı.
- 3. bölüm: Church-Turing tezi ve Turing makinelerine giriş
  - Dil hiyerarşisi: düzenli $\subseteq$ bağlamdan bağımsız $\subseteq$ Turing-tanınabilir.
  - Turing makinesinin yapısı: iki yönde sonsuz şerit, okuma-yazma kafası, durumlar.
  - Geçiş gösterimi $a \to b, L$ ve anlık betimleme $u\,q\,v$.
  - Deterministik olma, kısmi geçiş fonksiyonu, durma (halting), kabul ve ret.
  - Örnekler: a^* tanıyan makine, her şeyi kabul eden makine, sonsuz döngü örneği, $\{a^n b^n \mid n \geq 1\}$ (ev çalışması).

Hocanın Özellikle Vurguladığı Kısımlar

- Eşdeğerlik ispatlarında iki yön gösterilir: her iki kümenin de diğerinin alt kümesi olduğu.
- CFG'den PDA'ya dönüştürme kolaydır. PDA'nın yalnız üç durumu vardır ve iki geçiş her zaman aynıdır.
- Hesaplama ağacını her zaman yazmak zorunda değilsiniz. Çalışacak dalı görebiliyorsanız doğrudan onu yazın.
  - Hoca, $abab$ örneğini bütün ağacı yazmadan kendinizce denemenizi istedi.
- PDA'dan CFG'ye dönüştürme karmaşıktır ve çok büyük bir gramer üretir. Derste yapılmadı. İlgilenen kitaptan okusun.
- Kitaptaki alıştırmaları yapın.
- Turing makinesinin farkı: şeride **yazabilir**. Bu yüzden şerit bilgisayar belleği gibidir.
- Turing makineleri (aksi söylenmedikçe) deterministiktir. Ama geçiş fonksiyonu kısmi olabilir: Her durumda her sembol için geçiş çizmek gerekmez.
- Turing makinelerinde kabul durumlarından çıkan geçiş olmaz. Kabul durumuna gelen makine durur ve kabul eder.
- Turing makinesi girişin hepsini okumadan da kabul edebilir.
- Sonsuz döngü büyük bir sorundur. Bir makinenin durup durmayacağını genel olarak bilemeyiz (durma problemi, halting problem).

Kısa Tekrar Notları

- **CFG → PDA yapısı:**
  - $q_0 \xrightarrow{\varepsilon,\ \varepsilon \to S} q_1$ (S başlangıç değişkeni).
  - q_1'de döngü:
    - her $A \to w$ kuralı için $\varepsilon, A \to w$,
    - her a terminali için $a, a \to \varepsilon$.
  - $q_1 \xrightarrow{\varepsilon,\ \$ \to \$} q_2$, q_2 kabul durumu.
- **Sezgi:** Yığında en soldan türetimin henüz eşleşmemiş kısmı durur. Değişken tepedeyse kural uygulanır; terminal tepedeyse girişle eşleştirilip çekilir.
- **Teorem:** G, w'yu üretir ⇔ M, w'yu kabul eder. Dolayısıyla $L(G) = L(M)$. Bu bir yapıyla ispattır (proof by construction).
- **Dil hiyerarşisi:** Düzenli diller (FA, ör. a^*) ⊆ bağlamdan bağımsız diller (CFG, PDA, ör. $a^n b^n$) ⊆ Turing-tanınabilir diller (TM, ör. $a^n b^n c^n$).
- **TM geçişi:** $q_1 \xrightarrow{a \to b,\ L} q_2$: q_1'deyken kafanın altında a varsa b yaz, kafayı sola (L) taşı, q_2'ye geç. R sağ demektir.
- **TM'nin her adımı:** sembol oku, sembol yaz, sola ya da sağa git.
- **Başlangıç şeridi:** Giriş şeride yazılı, kafa girişin ilk sembolünde, geri kalan her yer boşluk sembolü (derste ⋄, kitapta ␣).
- **TM ID'si:** $abc\, q_0\, de$ = kafanın solunda abc, makine q_0'da, kafa d'nin üzerinde, sağında e.
- **Üç olası sonuç:**
  - Kabul durumunda durursa kabul.
  - Kabul olmayan bir durumda durursa ret.
  - Sonsuz döngüye girerse kabul etmez.

Detaylı Açıklamalar

PDA tekrarı

- ID üç parçalıdır: durum, okunmamış giriş, yığın içeriği. Durum diyagramı elinizdeyse makineyi her seferinde çizmeden ID'leri art arda yazarak hesaplamayı sürdürebilirsiniz.
- Deterministik olmama yüzünden bazı noktalarda hesaplama çoğalır ve bir ağaca dönüşür.
- Dizgi çekme/itme bir kısaltmadır. Örneğin $eb \to cdf$, önce e ve b'nin çekilmesi, sonra f, d, c'nin sırayla itilmesi demektir.
- 7'li tanımda δ'nın sonuçları kümedir. Aynı (durum, giriş, yığın tepesi) için farklı durumlara ya da farklı itme dizgilerine gidilebilir.
- Dil: başlangıç durumunda, girişin tamamı ve yalnız yığın başlangıç sembolüyle başlayıp, girişin tamamını tüketerek bir kabul durumuna ulaşılabilen dizgiler. Yığında ne kaldığı önemli değildir.

CFG'den PDA'ya dönüştürme

Bir G grameri için $L(G) = L(M)$ olan M şöyle kurulur:

1. $q_0 \to q_1$: $\varepsilon, \varepsilon \to S$. Başlangıç değişkeni yığına itilir.
2. q_1'de döngü:
   - Her üretim $A \to w$ için $\varepsilon, A \to w$: hiçbir şey okumadan tepedeki A'yı çek, w'yu it.
   - Her terminal a için $a, a \to \varepsilon$: girişten a oku, tepedeki a'yı çek.
3. $q_1 \to q_2$: $\varepsilon, \$ \to \$$. Yığında yalnız $\$$ kaldıysa kabul durumuna geç.

Örnek: $S \to aSTb \mid b$**,** $T \to Ta \mid \varepsilon$

q_1'deki altı geçiş:

Kaynak	Geçiş
$S \to aSTb$	$\varepsilon, S \to aSTb$
$S \to b$	$\varepsilon, S \to b$

Kaynak	Geçiş
$T \to Ta$	$\varepsilon, T \to Ta$
$T \to \varepsilon$	$\varepsilon, T \to \varepsilon$
terminal a	$a, a \to \varepsilon$
terminal b	$b, b \to \varepsilon$

$abab$ girişinin kabul eden dalı:

Adım	ID	Açıklama
0	$(q_0, abab, \$)$	
1	$(q_1, abab, S\$)$	S itildi
2	$(q_1, abab, aSTb\$)$	$S \to aSTb$
3	$(q_1, bab, STb\$)$	a eşleşti
4	$(q_1, bab, bTb\$)$	$S \to b$
5	$(q_1, ab, Tb\$)$	b eşleşti
6	$(q_1, ab, Tab\$)$	$T \to Ta$
7	$(q_1, ab, ab\$)$	$T \to \varepsilon$
8	$(q_1, b, b\$)$	a eşleşti
9	$(q_1, \varepsilon, \$)$	b eşleşti
10	$(q_2, \varepsilon, \$)$	kabul

Hesaplama ağacındaki diğer dallar:

- 2. adımda $S \to b$ uygulayan dal ölür: girişte a var, tepede b var, böyle bir geçiş yok.
- $T \to Ta$'yı tekrar tekrar uygulayan dal hiç giriş okumadan sonsuza kadar sürebilir. Ama bir dal kabul ettiği için dizgi kabul edilir.

PDA en soldan türetimi taklit eder

Kabul eden dalda yığın içerikleri sırasıyla: \$, S\$, $aSTb$\$, STb\$, bTb\$, Tb\$, Tab\$, ab\$, b\$, \$.

Gramerdeki en soldan türetim:

$$S \Rightarrow aSTb \Rightarrow abTb \Rightarrow abTab \Rightarrow abab$$

Yığın içerikleri, türetimdeki cümlesel biçimlerin aynısıdır. Tek fark, başta girişle eşleşmiş terminallerin yığından silinmiş olmasıdır. Yığının tepesi dizginin soluna karşılık gelir.

PDA'dan CFG'ye dönüştürme

Her PDA M için $L(M) = L(G)$ olan bir CFG G vardır. Hoca bu yapının karmaşık olduğunu ve çok büyük bir gramer ürettiğini söyleyerek atladı. İlgilenenler kitaptan okuyabilir.

Dil hiyerarşisi

- **Düzenli diller:** Sonlu otomatlar (DFA/NFA) tanır. Örnek: a^*.
- **Bağlamdan bağımsız diller:** CFG üretir, PDA tanır. Örnek: $a^n b^n$.
- **Turing-tanınabilir diller (Turing recognizable):** Turing makineleri tanır. Örnek: $a^n b^n c^n$.
- Her küme bir öncekini kapsar: $RL \subseteq CFL$. Hoca ayrıca bilgisayarla hesaplanabilen her şeyin bir Turing makinesiyle hesaplanabildiğini söyledi.

Turing makinesinin yapısı

- **Şerit:** İki yönde sonsuzdur. Başlangıçta giriş şeride yazılıdır, diğer bütün hücreler boşluk sembolüdür ($\diamond$).
- **Okuma-yazma kafası:** Başlangıçta girişin ilk sembolünü gösterir. Her adımda bir sembol okur, bir sembol yazar ve sola ya da sağa gider.
- **Kontrol birimi:** Durumlar, bir başlangıç durumu, bir ya da daha fazla kabul durumu ve geçişler.
- Kitapta şerit yalnız sağa doğru sonsuzdur. İki model denktir: Solda yer gerekirse her şey sağa kaydırılabilir.

Deterministiklik ve kısmi geçiş fonksiyonu

- Aynı durum ve aynı sembol için iki farklı geçiş olamaz. Örneğin q_1'de a için hem (q_2, b, R) hem (q_3, d, L) yazılamaz.
- ε (boş) geçiş yoktur.
- Deterministik olmayan Turing makineleri ayrı bir modeldir ve sonra görülecek.
- DFA'da her durum için alfabedeki her sembolün geçişi yazılmalıydı. Turing makinelerinde gerekmez: Geçiş fonksiyonu kısmi (partial) olabilir. Bunun amacı karmaşık diyagramları sadeleştirmektir.

Durma, kabul ve ret

- Makine, izleyebileceği bir geçiş yoksa o durumda **durur** (halts). Bu iki biçimde olur: Durumda hiç çıkan geçiş yoktur ya da kafanın altındaki sembole uyan geçiş yoktur.
- Kabul durumlarından çıkan geçiş yoktur. Bu yüzden kabul durumuna gelen makine durur ve kabul eder.
- Üç sonuç:
  - Kabul durumunda durursa giriş kabul edilir.
  - Kabul olmayan bir durumda durursa giriş reddedilir.
  - Sonsuz döngüye girerse giriş kabul edilmez.
- Turing makinesi girişi "tüketmez", yalnız değiştirir. Girişin tamamını taramadan kabul edebilir.

Örnek 1: a^* tanıyan makine

- q_0'da döngü: $a \to a, R$.
- $q_0 \to q_1$: $\diamond \to \diamond, L$. q_1 kabul durumudur.
- Giriş alfabesi $\{a, b\}$, ama diyagramda b için geçiş yok.
- aaa: Kafa her a'da sağa gider. Boşluğa gelince q_1'e geçer ve kabul eder.
- aba: $q_0\, aba \vdash a\, q_0\, ba$. q_0'da b için geçiş yok. Makine durur ve reddeder.

Örnek 2: Hiç taramadan kabul eden makine

Başlangıç durumu q_0 aynı zamanda kabul durumuysa, makine hiçbir şey okumadan durur ve kabul eder. Bu makine her dizgiyi kabul eder.

Örnek 3: Sonsuz döngü

- q_0'da döngü: $a \to a, R$ ve $b \to b, L$.
- $q_0 \to q_1$: $\diamond \to \diamond, L$. q_1 kabul durumudur.
- aba girişi:
  - Zaman 0: $q_0\, aba$
  - Zaman 1: $a\, q_0\, ba$
  - Zaman 2: $q_0\, aba$
  - Zaman 2, zaman 0'ın aynısıdır. Makine sonsuz döngüye girmiştir ve hiçbir zaman kabul durumuna ulaşmaz.
- Döngüyü, daha önce görülmüş bir ID'ye geri dönüldüğünü fark ederek yakalayabiliriz. Ama döngü çok uzun bir hesaplamadan sonra da başlayabilir. Genel olarak bir makinenin durup durmayacağını bilemeyiz. Buna durma problemi denir.

Örnek 4 (ev çalışması): $\{a^n b^n \mid n \geq 1\}$

Durum	Okunan	Yazılan, yön	Sonraki
q_0	a	x, R	q_1
q_0	y	y, R	q_3
q_1	a	a, R	q_1
q_1	y	y, R	q_1
q_1	b	y, L	q_2
q_2	y	y, L	q_2
q_2	a	a, L	q_2
q_2	x	x, R	q_0
q_3	y	y, R	q_3
q_3	$\diamond$	$\diamond, L$	q_4 (kabul)

Fikir: En soldaki a'yı x yap. Sağa gidip ilk b'yi y yap. Sola dönüp x'e kadar git. Tekrarla. a kalmayınca yalnız y'ler ve ardından boşluk varsa kabul et. Hoca $aabb$ girişiyle kendiniz denemenizi istedi.

Ders sonrası notlar

- **Hiyerarşi çıkarımları ters söylendi:** Hoca "Bir dilin bağlamdan bağımsız olduğunu söylerseniz otomatik olarak düzenli olduğunu ispatlamış olursunuz; düzenli olmadığını ispatlarsanız bağlamdan bağımsız olmadığını ispatlamış olursunuz" dedi. İkisi de yanlıştır. $RL \subseteq CFL$ olduğu için doğrusu:
  - Bir dil düzenliyse bağlamdan bağımsızdır.
  - Bir dil bağlamdan bağımsız değilse düzenli de değildir.
  - Örnek: $a^n b^n$ bağlamdan bağımsızdır ama düzenli değildir.
- **Sonsuz döngü örneğindeki not:** Hoca önce zaman 2'nin zaman 0'ın aynısı olduğunu doğru söyledi, ardından "zaman 2, zaman 1'in aynısıdır" dedi (02:39:33). Doğrusu: Zaman 2 ($q_0\, aba$), zaman 0'ın aynısıdır.
- a^* **makinesinde yön:** Hoca bir adımı anlatırken "yaz ve sola git" dedi (02:28:08). Geçiş $a \to a, R$ olduğu için doğrusu sağa gitmektir.
- **"Aynı dil" sözü:** Hoca bir sonraki makineye "aynı dili kabul eden daha basit bir makine" diye başladı (02:35:13). Bu makine a^*'ı değil bütün dizgileri kabul eder; hoca da hemen ardından bunu söyledi.
- **Durma problemi:** Hoca karmaşık bir makinenin durup durmayacağını göremeyeceğimizi söyledi. Kesin sonuç şudur: Durma problemi karar verilemezdir (undecidable). Her makine ve her giriş için doğru cevabı veren genel bir algoritma yoktur. Bu, karmaşık bir programın durduğunun hiçbir zaman gösterilemeyeceği demek değildir.
- **Sonsuz döngü makinesinin dili:** Bu makine b ile başlayan dizgileri kabul eder. Örneğin b girişinde kafa sola, boşluğa gider ve $\diamond \to \diamond, L$ ile q_1'e geçer. Makinenin kabul ettiği dil a^* ile b ile başlayan dizgilerin birleşimidir. ab ya da aba gibi bir a'dan sonra b gelen dizgilerde döngüye girer. Python simülasyonuyla doğrulandı.
- **"Sonsuz döngüde reddeder" ifadesi:** Hoca döngüdeki makine için "reddeder" dedi. Döngüye giren makine dizgiyi **kabul etmez**, ama durup reddetmiş de sayılmaz. Bu fark önemlidir: Her girişte duran makineye karar verici (decider) denir. Karar verici bir makinenin tanıdığı dillere karar verilebilir (decidable) diller denir. Yalnız tanıyan (recognizer) makine bazı girişlerde döngüye girebilir.
- **Kısmi geçiş fonksiyonu kuralı kaynağa bağlıdır:** Sipser'da δ tam bir fonksiyondur ve ayrı q_{accept}, q_{reject} durumları vardır. Tanımsız geçişler, örtük olarak ret durumuna gider. Dersteki gösterim (kısmi δ, birden fazla kabul durumu, geçişsiz durumda durma) Linz'in kitabındaki gösterimdir. Sonuç olarak iki model denktir.
- $\{a^n b^n\}$ **makinesinin alt sınırı:** Hoca sözlü olarak "n birden büyük" dedi (02:49:47). Makine ab'yi kabul ettiği için doğrusu $n \geq 1$'dir. Boş dizgiyi kabul etmez.
- Γ **ve** \$**:** Hoca yığın alfabesinin \$'ı içermediğini söylemiş görünüyor. \$ yığında durduğu ve geçişlerle çekilip geri itildiği için geçiş fonksiyonu onu yığın tepesi olarak görebilmelidir. Yalnız giriş alfabesinde yer almaz.

- **Doğrulama:** Örnek gramerin PDA'sı Python ile simüle edildi. 6 sembole kadar bütün $\{a, b\}$ dizgilerinde gramerle aynı dili tanıdığı doğrulandı. a^* makinesi ve $\{a^n b^n \mid n \geq 1\}$ makinesi de 8 sembole kadar bütün dizgilerde doğru çalıştı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz. Ders İngilizce anlatılmıştır; altyazı da büyük ölçüde İngilizcedir.